

```
class Part:
    def __init__(self, id, name, price, manufacturer_id):
        self.id = id
        self.name = name
        self.price = price
        self.manufacturer_id = manufacturer_id
```

```
class Manufacturer:
    def __init__(self, id, name):
        self.id = id
        self.name = name
```

```
class ManufacturerPart:
    def __init__(self, part_id, manufacturer_id):
        self.part_id = part_id
        self.manufacturer_id = manufacturer_id
```

```
manufacturers = [
    Manufacturer(1, "АвтоКомплект"),
    Manufacturer(2, "АльфаТех"),
    Manufacturer(3, "БетаМаш")
]
```

```
parts = [
    Part(1, "Колесо", 500, 1),
    Part(2, "Двигатель", 5000, 1),
    Part(3, "Фара", 300, 2),
    Part(4, "Коробка передач", 2500, 2),
    Part(5, "Шестерня", 100, 3)
]
```

```
manuf_parts = [
    ManufacturerPart(1, 1),
    ManufacturerPart(2, 1),
    ManufacturerPart(3, 2),
    ManufacturerPart(4, 2),
    ManufacturerPart(5, 3),
]
```

```

def query_manufacturers_starting_with_a():
    result = {}
    for manuf in manufacturers:
        if manuf.name.startswith('A'):
            parts_in_manuf = [part.name for part in parts if part.manufacturer_id == manuf.id]
            result[manuf.name] = parts_in_manuf
    return result

def query_manufacturers_with_max_part_price():
    result = {}
    for manuf in manufacturers:
        manuf_parts = [part for part in parts if part.manufacturer_id == manuf.id]
        if manuf_parts:
            max_part = max(manuf_parts, key=lambda part: part.price)
            result[manuf.name] = (max_part.name, max_part.price)
    return dict(sorted(result.items(), key=lambda x: x[1][1], reverse=True))

def query_all_related_parts_and_manufacturers():
    result = {}
    many_to_many_temp = [(part.name, mp.part_id, mp.manufacturer_id)
                          for part in parts
                          for mp in manuf_parts
                          if part.id == mp.part_id]

    many_to_many = [(part_name, part_id, manuf.name)
                    for part_name, part_id, manuf_id in many_to_many_temp
                    for manuf in manufacturers if manuf.id == manuf_id]

    for manuf in manufacturers:
        related_parts = list(filter(lambda x: x[2] == manuf.name, many_to_many))
        result[manuf.name] = related_parts
    return result

def main():
    print("=== Запрос 1: Производители с названиями на 'A' ===")
    for manufacturer, parts in query_manufacturers_starting_with_a().items():
        print(f"{manufacturer}: {' '.join(parts)}")

    print("\n=== Запрос 2: Производители с самой дорогой деталью ===")
    for manufacturer, (part_name, price) in query_manufacturers_with_max_part_price().items():
        print(f"{manufacturer}: {part_name} (Цена: {price})")

    print("\n=== Запрос 3: Связанные производители и детали ===")
    for manufacturer, parts in query_all_related_parts_and_manufacturers().items():
        part_list = ', '.join([f"{p[0]} ({p[1]})" for p in parts])
        print(f"{manufacturer}: {part_list}")

if __name__ == "__main__":
    main()

```

```
C:\Users\brofo\PycharmProjects\TheDota2Predictor\venv\Scripts\p
=== Запрос 1: Производители с названиями на 'А' ===
АвтоКомплект: Колесо, Двигатель
АльфаТех: Фара, Коробка передач

=== Запрос 2: Производители с самой дорогой деталью ===
АвтоКомплект: Двигатель (Цена: 5000)
АльфаТех: Коробка передач (Цена: 2500)
БетаМаш: Шестерня (Цена: 100)

=== Запрос 3: Связанные производители и детали ===
АвтоКомплект: Колесо (1), Двигатель (2)
АльфаТех: Фара (3), Коробка передач (4)
БетаМаш: Шестерня (5)

Process finished with exit code 0
```